2024 年同等学力申硕软件工程考前必备资料

一、软件过程和软件工程

软件过程是指在软件开发和维护过程中，按照一定的方法和步骤来组织、管理和执行软件项目的活动。它涵盖了项目规划、需求分析、设计、编码、测试、部署和维护等阶段，并包括与项目相关的管理和控制活动。

软件工程是一门工程学科，关注如何以系统化、规范化和可靠的方式开发和维护软件。它涵盖了软件开发的全过程，包括需求分析、软件设计、编码、测试、部署和维护等阶段，并强调使用工程原则和方法来管理和控制软件项目。

二者区别如下：

1，范围：软件过程是指软件项目中的活动和过程，而软件工程是一门学科，研究如何以工程化的方式进行软件开发和维护。

2，目标：软件过程的目标是按照一定的步骤和方法来组织和执行软件项目，以达到项目交付的要求。软件工程的目标是通过应用工程原则和方法来提高软件开发的效率、质量和可靠性。

3，范畴：软件过程关注具体的项目执行和管理，包括项目规划、需求分析、设计、编码、测试等活动。软件工程更广泛，包括软件开发的全过程，以及与软件开发相关的工具、方法、标准和最佳实践等。

4，视角：软件过程更侧重于操作层面，关注具体的项目活动和流程。软件工程更侧重于理论和方法的研究，关注软件开发的原则、模型、方法和工具。

5，关系：软件过程是软件工程的一部分，是软件工程理论和方法在实际项目中的应用。软件工程提供了一系列的原则、模型和方法，用于指导和改进软件过程的执行和管理。

总的来说，软件过程是软件工程的实践，是软件开发项目中具体的活动和流程。而软件工程是一门学科，研究如何以工程化的方式开发和维护软件，并提供相应的原则、模型和方法。

二、软件工程开发模型

软件开发过程中采用的一种组织和管理方法。它们提供了一套结构化的步骤和指导原则，用于指导软件项目的规划、设计、开发、测试和交付。

1，瀑布模型（Waterfall Model）：

瀑布模型是一种线性顺序的开发模型，将软件开发过程划分为一系列连续的阶段，如需求分析、系统设计、编码、测试和维护。

每个阶段在前一个阶段完成之后开始，阶段之间存在严格的依赖关系和交付物。

强调阶段之间的一致性和文档化，适用于需求稳定、对变更要求较低的项目。

瀑布模型特点：

1） 阶段间具有顺序性和依赖性

2） 推迟实现的观点

3） 质量保证的观点

虽然瀑布模型是一个比较“老”的、甚至过时的开发模型，但其优点为：在决定系统怎样做之前，存在一个需求阶段，鼓励对系统“做什么”进行规约（即设计之前的规约）。

在建造构件之前，存在一个设计阶段，鼓励规划系统结构（即编码之前的设计）。

在每一阶段结束时进行复审，允许获取方和用户的参与。

允许基线和配置早期接受控制。

前一步工作产品可作为下一步被认可的、文档化的基线。

瀑布模型存在的不足：

客户必须能够完整、正确和清晰地表达他们的需求；开发人员一开始就必须理解其应用。在开始的两个或三个阶段中，很难评估真正的进度状态；设计、编码和测试阶段都可能发生延期。在一个项目的早期阶段，过分地强调了基线和里程碑处的文档；可能要花费更多的时间，用于建立一些用处不大的文档。当接近项目结束时，出现了大量的集成和测试工作。直到项目结束之前，都不能演示系统的能力。

瀑布模型适用的情况：

在开发中，向下、渐进的路径占支配地位。也就是说，需求已被很好地理解；

并且过程设计人员也很清楚：开发组织非常熟悉为实现这一模型所需要的过程（或经过培训后，熟悉什么时候来支持这一项目，以实现这一模型所需要的过程。

因此为了避免产生过多的反复迭代工作，增加开发成本，一般在准备采用瀑布模型(也包括其他模型)时，需要考虑以下 2 个问题：

一是过程设计人员必须对初始产品(通常是软件需求规约，SRS)的不确定性进行评估。

二是组织是否具有熟练实施每个活动和任务的历史经验。

2，增量模型（Incremental Model）：

增量模型将软件开发过程划分为多个增量，每个增量都是一个完整的开发周期，包括需求分析、设计、开发和测试。

每个增量都会增加新的功能或改进现有功能，通过逐步迭代的方式逐渐构建完整的系统。

适用于大型项目、需求不断变化的环境，能够快速响应变化和提供可部署的增量产品。

3，原型模型（Prototype Model）：

原型模型通过创建初始版本的软件原型来理解用户需求、验证设计概念和获取反馈。

原型可以是低保真的草图、模拟界面或具有基本功能的快速开发版本。

用户的反馈和洞察力被用于改进和迭代原型，直到最终满足需求。

适用于需求不清晰、用户参与度高、创新性强的项目。

4，敏捷模型（Agile Model）：

敏捷模型是一种迭代增量开发的方法，强调团队协作、快速响应变化和持续交付价值。

开发过程通过短期迭代周期（如 Scrum 中的 Sprint）来组织，每个迭代产生一个可交付的软件增量。需求和解决方案在迭代中不断演化，团队通过反馈和适应来持续改进和优化。

适用于需求变化频繁、强调快速交付和灵活性的项目。

5，螺旋模型（Spiral Model）：

螺旋模型将风险管理作为核心，将软件开发过程划分为多个循环（每个循环称为一个螺旋）。每个螺旋包括需求分析、风险评估、原型开发、评审和计划，通过迭代的方式逐步构建系统。风险驱动的方法使团队能够在早期识别和解决问题，减少项目失败的风险。

适用于复杂、关键和高风险的项目。

这些软件工程开发模型各有其特点和适用场景。团队可以根据项目需求、时间、资源和风险因素选择最合适的开发模型，以帮助实现高质量的软件交付。

定量管理级主要特征：

项目：项目减小过程性能的变化性，使其进入可接收的量化边界，从而达到对产品和过程的控制。

组织：为软件产品和过程都设定了量化的质量目标。

过程能力：可预言的。(predictable)

三、软件需求与软件需求规约

1 定义问题的基本要素是”需求”

1) 需求：一个需求是一个有关“要予构造”的陈述，用以描述待开发产品（或项）功能上的能力、性能参数或者其它性质。

2)需求的分类：功能需求、性能需求、外部接口需求、设计约束、质量属性

3)需求获取的方法：访谈、工作坊、焦点小组、观察、问卷调查、系统接口分析、用户界面分析、文档分析

4)需求规约格式实例

5)表达需求规约(规格说明书)的三种风格

1，自然语言风格是最常见和直观的表达需求规约的方式。它使用自然语言，类似于我们日常交流的方式，以清晰、简洁的语句描述需求规约。这种风格更接近人类的语言表达方式，易于理解和阅读。然而，自然语言风格可能存在歧义、模糊性和解释性差的问题，因此在编写规约时需要特别注意准确性和一致性。

2，结构化风格使用结构化的模板和符号来表达需求规约。它使用特定的格式和规则，例如用例模板、状态转换图、活动图等，以更系统化和一致的方式描述需求规约。结构化风格能够提供更清晰的结构和组织，有助于减少歧义和模糊性，并支持自动化工具的使用和分析。

3，形式化风格是最严格和精确的表达需求规约的方式。它使用形式化的符号、逻辑和数学语言来描述需求规约，以确保规约的准确性和一致性。形式化风格通常使用形式化规约语言，如 Z 语言、时序逻辑、Petri 网等。这种风格适用于对系统行为和约束进行严格推理和验证的场景，但需要具备形式化方法的专业知识和技能。

6)需求规约的作用

四、结构化方法学

1.结构化设计方法

1)设计：一种软件开发活动,定义实现需求规约所需结构.

设计目标:依据需求规约,在一个抽象层上建立系统软件模型,包括软件体系结构（数据和程序结构），以及详细的处理算法,产生设计规格说明书.

2)对设计方法的需求

a) 提供可体现”原理/原则”的一组术语(符号),形成一个特定的抽象层,用于表达设计中所使用的部件.

b) 依据术语所形成的”空间”,给出表达软件模型工具.

c) 给出设计的过程指导.

3)结构化设计方法

（1）在总体设计层:

(a)引入了两个术语/符号

模块:一种可独立标识的软件成分.

调用:模块间的一种关系,模块 A 为了完成其任务必须依赖其他模块.

(b)引入了模块结构图:用于表达软件系统的静态结构.

2.内聚:一个模块内部各个元素彼此结合的紧密程度。它是衡量一个模块内部组成部分间整体统一性的度量。常见的内聚有七类。

1）功能内聚（Functional Cohesion）:如果一个模块内所有处理元素完成一个，而且仅完成一个功能，则称为功能内聚。功能内聚是最高程度的内聚。但在软件结构中，并不是每个模块都能设计成一个功能内聚模块。

2）顺序内聚（Sequential Cohesion）:如果一个模块内处理元素和同一个功能密切相关，而且这些处理元素必须顺序执行，则称为顺序内聚。

3）通信内聚（Communicational Cohesion）:如果一个模块中所有处理元素都使用同一

个输入数据和（或）产生同一个输出数据，称为通信内聚。

4）过程内聚（Procedural Cohesion）:如果一个模块内的处理元素是相关的，而且必须以特定的次序执行，称为过程内聚。

注意：过程内聚与顺序内聚的区别是：顺序内聚中是数据流从一个处理单元流到另一个处理单元，而过程内聚是控制流从一个动作流向另一个动作。

5）时间内聚（Temporal Cohesion）：如果一个模块包含的任务必须在同一段时间内执行，称为时间内聚。也称为瞬时内聚。

6）逻辑内聚（Logical Cohesion）：如果模块完成的任务在逻辑上属于相同或相似的一类，称为逻辑内聚。

7）偶然内聚（Coincidental Cohesion）：如果一个模块由完成若干毫无关系的功能处理元素偶然组合在一起的，就叫偶然内聚。

3.详细设计阶段的任务：

1）为每个模块确定采用的算法，选择某种适当的工具表达算法的过程，写出模块的详细过程性描述；

2）确定每一模块使用的数据结构；

3）确定模块接口的细节，包括对系统外部的接口和用户界面，对系统内部其它模块的接口，以及模块输入数据、输出数据及局部数据的全部细节。

4）要为每一个模块设计出一组测试用例，以便在编码阶段对模块代码（即程序）进行预定的测试，模块的测试用例是软件测试计划的重要组成部分，通常应包括输入数据，期望输出等内容。

五、面向对象方法学

（一）用例之间的关系

扩展关系：向一个用例中添加一些动作后构成另一个用例，它们之间构成扩展关系。

使用关系：一个用例使用另一个用例，它们之间构成使用关系。

（二）UML 定义重要内容的五类图：

1）用例图：从用户角度描述系统功能，并指出各功能

2）静态图：包括类图、对象图和包图；

3）行为图：描述系统的动态模型；

4）交互图：描述对象间的交互关系；

5）实现图：如配置图，定义系统中软硬件的物理体系结构

（三）聚集

聚集表示类与类之间是整体与部分的关系。

共享聚集（松散聚集）：处于部分方的对象可同时参与多个处于整体方对象的构成。

组合聚集（紧密聚集）：部分类完全隶属于整体类，部分与整体共存，整体不存在了部分也随之消失。

（四）动态模型的组成

1）事件：事件是某个特定时刻所发生的事情。它是引起对象状态转换的控制信息。

2）状态：状态就是对象在其生命周期中的某个特定阶段所处的某种情形。

3）行为：行为是指对象达到某种状态时所做的一系列处理操作。

（五）功能模型

通常，功能模型由一组数据流图组成。UML 提供的用例图也是进行需求分析和建立功能模型的有力工具。以用例图建立起来的系统模型称为用例模型，它描述的是外部行为者所理解的系统功能。

六、面向数据结构的方法

（一）Jackson 结构程序设计方法步骤：

1）分析并确定输入数据和输出数据的逻辑结构，并用 Jackson 图描绘这些数据结构；

2）找出输入数据结构和输出数据结构中有对应关系的数据单元；

3）用三条规则从描绘数据结构的 Jackson 图导出描绘程序结构 Jackson 图：

4）列出所有操作和条件（包括分支条件和循环结束条件），并且把它们分配到程序结构图的适当位置；

5）用伪码表示程序。

（二）软件测试的步骤

1）模块测试：模块测试又称单元测试，它把每个模块作为单独的实体来测试。

2）子系统测试：子系统测试是把经过单元测试的模块放在一起形成一个子系统来测试。

3）系统测试：系统测试是把经过测试的子系统装配成一个完整的系统来测试。

4）验收测试：验收测试把软件系统作为单一的实体进行测试（利用用户的实际数据测试）。软件规格说明书是进行确认测试的基础。

5）平行运行：平行运行是同时运行新开发出来的系统和将被它取代的旧系统，以便比较新旧两个系统的处理结果。

七、软件维护

软件维护是软件生命周期的最后一个阶段。是指在软件产品发布后，因修正错误、提升性能或其他属性而进行的软件修改。

（一）软件维护的类型

1）改正性维护：交付给用户使用的软件，即使通过严格的测试，仍可能有一些潜在的错误在用户使用的过程中发现和修改。诊断和改正错误的过程称为改正性维护。

2）适应性维护：随着计算机的飞速发展，新的硬件系统和外部设备时常更新和升级，一些数据库环境、数据输入/输出方式、数据存储介质等也可能发生变换。为了使软件适应这些环境变化而修改软件的过程叫做适应性维护。

3）完善性维护：在软件投入使用过程中，用户可能还会有新的功能和性能要求，可能会提出增加新功能、修改现有功能等要求。为了满足这类要求而进行的维护称为完善性维护。在各类维护中，完善性维护占软件维护工作的大部分。

4）预防性维护：为了改进软件未来的可维护性或可靠性，或者为了给未来的改进奠定更好的基础而进行的修改，称为预防性维护。这种维护活动在实践中比较少见。

（二）软件质量特征以及软件质量保证

软件质量：与软件产品满足规定的和隐含的需求的能力有关的特征或特性的全体。软件的质量因素定义如下：

1）正确性：软件满足设计规格说明及用户预期目标的程度，它要求软件没有错误。

2）可靠性：软件按设计要求，在规定时间和条件下不出故障、持续运行的程度。

3）效率：为完成预定功能，软件系统所需的计算机资源的多少。

4）完整性：为某一目的而保护数据，避免它受到有意、偶然的破坏或遗失的能力。

5）可使用性：用户学习、使用软件以及为程序准备输入和解释输出所需工作量的大小。

6）可维护性：对一个已投入运行的软件进行相应诊断和修改所需工作量的大小。

7）可测试性：测试软件以确保其能够执行预定功能所需工作量的大小。

8）灵活性：修改或改进一个已投入运行的软件所需工作量的大小。

9）可移植性：将一个软件系统从一个计算机系统移植到另一个计算机系统或环境所需工作量的大小。

10）复用性：一个软件（或软件的部件）能再次用于其它应用的程度。

11）互连性：连接一个软件或其他系统所需工作量的大小。